TASK 3: INTEGRATE THE WEB SEARCH AGENT

Project: AI-Based Startup Idea Validator with Market Analysis Assistance

Purpose
Build a small Python component that accepts a startup idea, creates focused search queries, retrieves web results through the Tavily Search API, and displays the titles, summaries, and source links.

Scope
This document covers Task 3 only. It is a Web Search Agent prototype, not the full multi-agent application. The example code must be run locally with a valid Tavily API key to confirm live results.

What the finished component does

• Accepts a startup idea from the user.

• Creates separate queries for market demand, competitors, and opportunities/challenges.

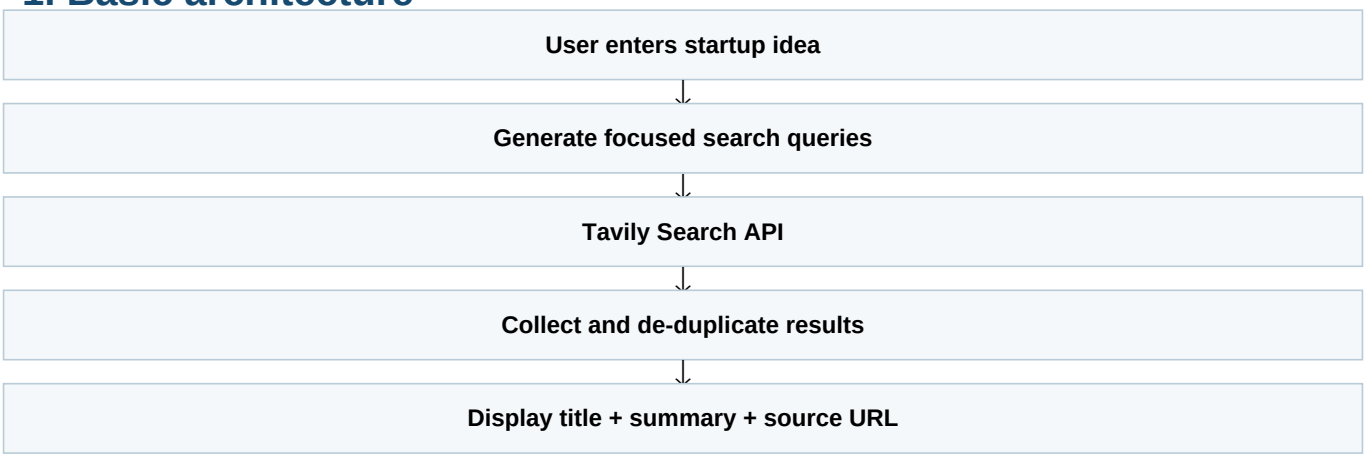
• Calls Tavily using its supported Python client and search method.

• Displays each result's title, short content summary, and source URL.

• Keeps the API key in an environment variable rather than hard-coding it.

• Handles empty input, missing API keys, and search errors.

1. Basic architecture



2. Tools and dependencies

Item	Purpose
Python 3.10+	Runs the search agent code.
tavily-python	Python SDK for the Tavily Search API.
TAVILY_API_KEY	Secret API key read from an environment variable.

3. Complete beginner-friendly Python code

Save the following code as **web_search_agent.py**. It uses the Tavily Python SDK pattern `TavilyClient(api_key=...)` and `client.search(query=...)`.

```
import os
import sys
from tavily import TavilyClient


def generate_search_queries(startup_idea):
    """Create focused market and competitor queries."""
    idea = startup_idea.strip()
    if not idea:
        return []

    return [
        f"{idea} target customers market demand industry trends",
        f"{idea} competitors alternatives similar businesses",
        f"{idea} market opportunities challenges pricing",
    ]


def search_startup_idea(startup_idea, max_results=4):
    """Search Tavily and return unique, readable results."""
    if not startup_idea or not startup_idea.strip():
        raise ValueError("Please enter a startup idea.")

    api_key = os.getenv("TAVILY_API_KEY")
    if not api_key:
        raise EnvironmentError(
            "TAVILY_API_KEY is missing. Set it before running this program."
        )

    client = TavilyClient(api_key=api_key)
    queries = generate_search_queries(startup_idea)
    results = []
    seen_urls = set()
    warnings = []

    for query in queries:
        try:
            response = client.search(
                query=query,
                search_depth="basic",
                max_results=max_results,
            )

            for item in response.get("results", []):
                url = item.get("url", "")
                if not url or url in seen_urls:
                    continue

                seen_urls.add(url)
                results.append({
                    "query": query,
                    "title": item.get("title") or "Untitled result",
                    "summary": item.get("content") or "No summary available.",
                    "url": url,
                })

        except Exception as error:
            # Keep going if one query fails.
            warnings.append(
                f"Search failed for query '{query}': {type(error).__name__}"
            )

    return queries, results, warnings


def main():
    print("AI Startup Idea Validator - Web Search Agent")
    idea = input("Enter your startup idea: ").strip()

    try:
        queries, results, warnings = search_startup_idea(idea)

        print("\\nSEARCH QUERIES")
        for number, query in enumerate(queries, start=1):
            print(f"{number}. {query}")

        print("\\nWEB RESULTS")
        if not results:
            print("No results returned. Check your key, connection, or query.")

        for number, item in enumerate(results, start=1):
            print("\\n" + "-" * 60)
            print(f"{number}. {item['title']}")
```

```
            print(f"Summary: {item['summary']}")
            print(f"Source: {item['url']}")
            print(f"Query: {item['query']}")

        if warnings:
            print("\\nWARNINGS")
            for warning in warnings:
                print(warning)

        print("\\nReview the source pages before relying on their claims.")

    except ValueError as error:
        print(f"Input error: {error}")
    except EnvironmentError as error:
        print(f"Configuration error: {error}")
    except Exception as error:
        print(f"Unexpected error: {type(error).__name__}: {error}")
        print("Check package installation, API key, and internet connection.")


if __name__ == "__main__":
    main()
```

4. Setup and execution

1. **Install Python:** use Python 3.10 or newer if available.
2. **Create a project folder:** for example, `Task3_Web_Search_Agent`.
3. **Save the code:** create `web_search_agent.py` and paste the code above.
4. **Install the SDK:** open a terminal in that folder and run `python -m pip install tavily-python`.
5. **Get a key:** sign up at Tavily's website and copy your API key from its dashboard.
6. **Set the environment variable:** use one of the commands below in the same terminal where you will run Python.
7. **Run:** execute `python web_search_agent.py`.

Set the API key

Terminal	Command (replace placeholder)
Windows Command Prompt	`set TAVILY_API_KEY=your_actual_key_here`
Windows PowerShell	`$env:TAVILY_API_KEY="your_actual_key_here"`
macOS / Linux	`export TAVILY_API_KEY="your_actual_key_here"`

Security: never paste your real API key into the Python source code or publish it in GitHub. If you later create a `.env` file, add it to `.gitignore`.

5. Example test and expected behaviour

Sample startup idea: “An app that delivers affordable homemade lunches to college students.”

Example query categories:

• Market: target customers, market demand, and industry trends.

• Competitors: similar businesses, alternatives, and services.

• Opportunities/challenges: pricing, customer needs, and potential barriers.

The exact search results will change over time and depend on the API response. Do not write invented results into your submission as if the program returned them. Run the program and capture your actual output.

6. Error handling

Case	Handling
Empty startup idea	The program displays an input error before searching.
Missing API key	The program explains that TAVILY_API_KEY must be set.
One search fails	The agent records a warning and continues with the other queries.
No results	The program displays a message asking the user to check the key, connection, or query.
Other API/runtime errors	A concise error type is displayed; check the terminal and configuration.

7. How this meets the Task 3 requirements

Task requirement	Implementation
Set up Tavily API	Uses the official Tavily Python client package.
Generate relevant queries	generate_search_queries() creates market, competitor, and opportunity queries.
Retrieve live information	search_startup_idea() calls the Tavily search endpoint.
Readable results	Displays result title, summary, URL, and related query.
Secure API key	Reads TAVILY_API_KEY from the process environment.
Handle errors	Validates empty input and missing key, and continues if an individual query fails.

8. Limitations and next step

This component gathers web evidence; it does not itself prove that a startup will succeed or calculate a verified market size. Open source links to check their relevance, date, and reliability. The simple query generator uses templates, not an LLM. A later milestone can connect this agent to the LLM-based idea extraction and the full multi-agent workflow.

9. References

• **Tavily official Python SDK repository**
https://github.com/tavily-ai/tavily-python

• **Tavily Python SDK search documentation**
https://github.com/tavily-ai/skills/blob/main/tavily-best-practices/references/search.md

• **Python documentation: environment variables via os.environ**
https://docs.python.org/3/library/os.html